



MySQL - RDBMS

Trainer: Nisha Dingare



- **1. Importance of Data Management**

- Every enterprise application needs to manage and organize data effectively.
- Efficient data handling ensures smooth business operations and decision-making.

- **2. Early Days of Data Storage**

- Initially, programmers stored data in simple files and performed operations manually.
- Data was **highly application-specific** and lacked standardization.
- Each application used its own **custom file format** (e.g., Tally, Address Book).

- **3. Evolution of Database Systems**

- As data management needs grew, **Database Management Systems (DBMS)** were introduced.
- **DBMS allowed developers to focus on business logic** instead of file handling.
- Popular early systems: **FoxPro, dBase, Excel**, etc.



Limitations of Traditional File-Based Databases :

- **1. File-Based and Less Secure**

- Data was stored in plain text or binary files without encryption.
- Anyone with file access could read or modify the data directly.
- No user authentication or role-based access control.

- **2. Single-User Systems**

- Designed for **individual use**, not for multiple users accessing data simultaneously.
- Multi-user access often caused **data inconsistency** and **corruption**.

- **3. Non-Distributed Architecture**

- Data was stored on a **local machine** — no sharing or synchronization across networks.
- Collaboration between departments or remote access was difficult.

- **4. Limited Data Capacity**

- Could handle only **small datasets** (usually a few megabytes).
- Not suitable for enterprise-scale data management or analytics.

- **5. Complicated Relationship Management**

- Relationships between files (tables) had to be **manually coded** by developers.
- No automatic **referential integrity** or relational features like in modern RDBMS.

- **6. File Locking Issues**

- When one user or process accessed a file, others were often **blocked**.
- This led to **performance bottlenecks** and **data access delays**.

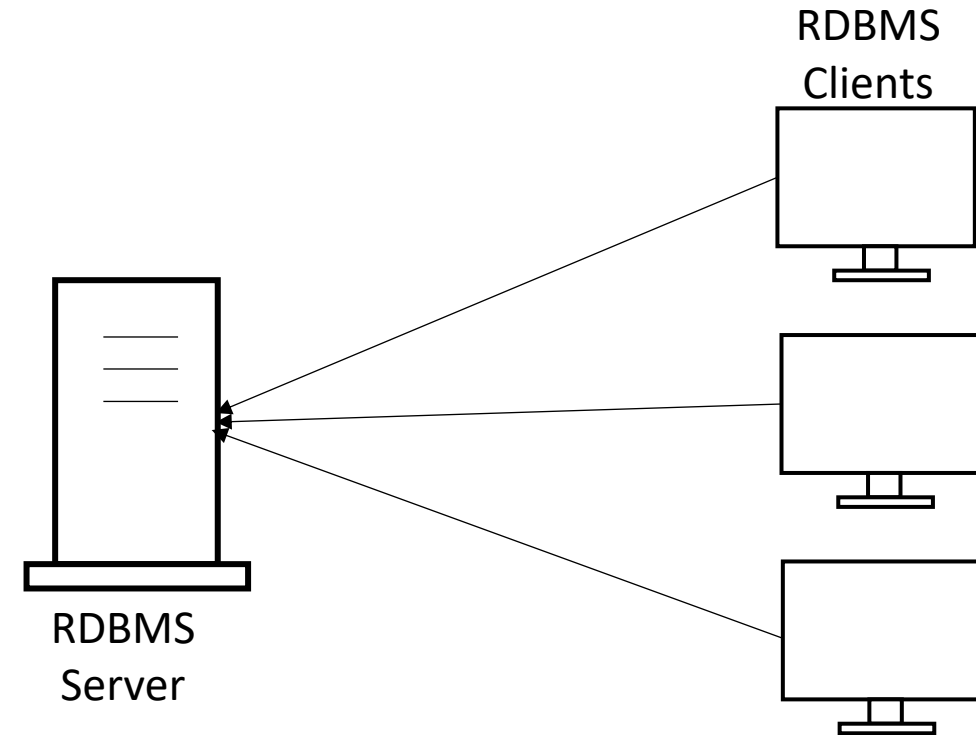
- **7. High Coding Effort**

- Developers had to write **many lines of code** to perform simple tasks (like searching, sorting, or linking data).
- Increased **development time** and higher chance of **errors** or **data loss**.



RDBMS

- **1. Definition :**
- **RDBMS** stands for **Relational Database Management System**.
- Organizes data into **tables (relations)** made up of **rows and columns**.
- Tables are **interrelated**, allowing easy data retrieval and management.
- **2. Key Features :**
- **Follows a structured, tabular format** for storing and organizing data.
- **Highly secure** with user access control and authentication.
- Supports **multi-user environments** for concurrent access.
- **Uses client-server architecture with server-side processing.**
- Can **manage large volumes of data (terabytes)** efficiently.
- Offers **built-in relational capabilities** (joins, relationships, constraints).
- Uses **table-locking or row-locking** to ensure data consistency.
- **Easily integrates** with various applications and platforms.
- **Eg: DB2, Oracle, MS SQL Server, MySQL, MS Access, SQLite, etc.**
- **RDBMS design is based on Codd's Rules, developed by Dr. E. F. Codd at IBM (1970).**



SQL : Structured Query Language

- **SQL (Structured Query Language)** is used to **communicate with and manage RDBMS** (Relational Database Management Systems).
 - Clients send **SQL queries** to the RDBMS server, which executes operations such as data creation, modification, retrieval, and deletion.
- **Standardization:**
 - Adopted by **ANSI (1986)** and **ISO (1987)** as a global standard.
 - Regularly **updated** to include new features and performance improvements.
 - **Latest revision: 2023.**
- **SQL is case-insensitive**, though **uppercase** is often used for keywords (e.g., SELECT, FROM).
- Works across **multiple platforms** (e.g., Oracle, MySQL, SQL Server, PostgreSQL, SQLite).
- **Naming Rules and Conventions :**
 - Names can include **letters (A–Z, a–z)**, **digits (0–9)**, and **limited special symbols (_ , \$, #)**.
 - Names **cannot begin with a digit**.
 - If a name contains special characters or spaces, enclose it in **backquotes (` )** or **double quotes ( " )** (depending on the RDBMS). Example - `SELECT `Employee#ID` FROM `Staff Records`;`
 - Examples: `Tbl1`, ``T1#``, ``T2$``
 - **Maximum length: 30 characters** (varies by database system).
 - Identifier length (name of a table/column):
 - Oracle → max 30 characters
 - MySQL → max 64 characters
 - Use **clear and meaningful names** for tables and columns for better maintainability.



SQL Commands :

- 2. Major Categories of SQL Commands

Category	Full Form	Examples	Purpose / Description
DDL	Data Definition Language	CREATE, ALTER, DROP, RENAME, TRUNCATE	Defines or modifies the structure of database objects like tables, views, and schemas.
DML	Data Manipulation Language	INSERT, UPDATE, DELETE, MERGE	Used to manipulate (add, modify, delete) the data stored in tables.
DQL	Data Query Language	SELECT	Retrieves and filters data from one or more tables.
DCL	Data Control Language	CREATE USER, GRANT, REVOKE	Controls user access , permissions , and security of the database.
TCL	Transaction Control Language	SAVEPOINT, COMMIT, ROLLBACK, SET TRANSACTION	Manages transactions , ensuring data consistency and allowing recovery from errors.



MySQL

- **1. Introduction**

- Developed by **Michael Widenius in 1995.**
- The name “**MySQL**” combines his daughter’s name “**My**” and the acronym “**SQL.**”
- One of the world’s most popular **Relational Database Management Systems (RDBMS)**, known for speed, reliability, and simplicity.

- **2. Ownership and History**

- **2008: MySQL was acquired by Sun Microsystems.**
- **2010: Oracle Corporation acquired Sun Microsystems, and consequently MySQL.**
- Despite being owned by Oracle, **MySQL remains open-source** and continues to have a strong global community.

- **3. Technical Features**

- Implements **client-server architecture** using standard **SQL interface**.
- Cross-platform support — available for **Windows, Linux, and macOS**.
- Offers **replication, partitioning, backup, and high-availability** features.

- **4. Related Technologies**

- **MariaDB**

- Developed by **Michael Widenius**, the original creator of MySQL.
- A **completely open-source fork of MySQL**, created after Oracle’s acquisition to preserve open development.
- **Fully compatible** with MySQL in terms of syntax, APIs, and data formats.

- **5. Editions of MySQL**

- **MySQL Community Server**

- **Free and open-source** edition.
- Suitable for developers, learners, and small-scale applications.

- **MySQL Enterprise Edition**

- **Commercial edition** offered by Oracle.
- Includes **advanced features** such as enhanced security, scalability, backup tools, and technical support.

- **6. Recognition**

- Named “**Database of the Year (2019)**” by **DB-Engines Ranking**.



Getting started

- root login can be used to perform CRUD as well as admin operations.

- It is recommended to create users for performing non-admin tasks.

- mysql> CREATE DATABASE db;
- mysql> SHOW DATABASES;
- mysql> CREATE USER dbuser@localhost IDENTIFIED BY 'dbpass';
- Mysql> USE db;
- mysql> SELECT user, host FROM mysql.user;
- mysql> GRANT ALL PRIVILEGES ON db.* TO dbuser@localhost;
- mysql> FLUSH PRIVILEGES;
- mysql> EXIT;

Accessing table and database names is not case-sensitive on Windows, but it is case-sensitive on Linux.

GRANT → give permission

ALL PRIVILEGES → give all permissions (SELECT, INSERT, UPDATE, DELETE, CREATE, etc.)

AC_classwork_db.* → all tables/objects inside AC_classwork_db

TO sunbeam → give these permissions to user sunbeam

AC_classwork_db.* → all tables in the database
AC_classwork_db.student → only the student table
*. * → all databases + all tables

- terminal> mysql -u dbuser -pdbpass

- mysql> SHOW DATABASES;
- mysql> SELECT USER(), DATABASE();
- mysql> USE db;
- mysql> SHOW TABLES;
- mysql> CREATE TABLE student(id INT, name VARCHAR(20), marks DOUBLE);
- mysql> INSERT INTO student VALUES(1, 'Abc', 89.5);
- mysql> SELECT * FROM student;



Database logical layout

- A Database (also called a Schema) is a logical container or namespace that stores all database objects related to a specific project or application.
- It helps organize data and associated objects in a structured and manageable way.
- It can contain various database objects, such as Tables, Constraints, Relationships, Stored Procedures, Functions, Triggers.
- Most RDBMSs create a few system databases automatically for internal use.
- In MySQL, the following system databases exist:
 - `mysql` – Stores user accounts, privileges, and authentication data.
 - `performance_schema` – Contains performance metrics and execution statistics.
 - `information_schema` – Stores metadata about databases, tables, columns, and constraints.
 - `sys` – Provides simplified views for performance tuning and analysis.
- Example: `SELECT user, host FROM mysql.user;`
- A database contains one or more tables.
 - Each table has multiple columns (fields).
 - Each column has a specific data type (e.g., INT, VARCHAR, DATE).
 - Columns may include constraints (e.g., NOT NULL, UNIQUE, DEFAULT).
 - Data in a table is stored in rows (records).
 - Each row holds values corresponding to each column in the table.
- Hierarchy: Database → Tables → Columns → Rows



Database physical layout

1. Data Storage Location

- In MySQL, data is stored on disk within the data directory, typically:
C:\ProgramData\MySQL\MySQL Server 8.4\Data
- Each database/schema corresponds to a separate sub-directory inside the data directory.

2. Table Storage

- Each table within a database is stored as one or more files on disk.
- For example, a table named `student` in the database `db` is stored as:
"C:\ProgramData\MySQL\MySQL Server 8.4\Data\ac_classwork_db\students.ibd"
- Data is stored in binary format for efficiency.

3. Data Layout on Disk

- Files are not necessarily stored contiguously on the physical hard disk.
- Data rows within a table file are scattered and not stored sequentially on disk.
- However, all fields (columns) within a single row are stored consecutively in the file.

4. Data Access

- When records are retrieved (selected), they can be returned in any order, depending on the query and indexing.
- MySQL uses indexes and storage engines to efficiently locate and access scattered data.



MySQL data types : Numeric : Integers

RDBMS have **similar but not identical** data types.

1. Numeric Types (Integers)

Data Type	Storage Size	Description
TINYINT	1 byte	Small integer
SMALLINT	2 bytes	Small integer
MEDIUMINT	3 bytes	Medium integer
INT	4 bytes	Standard integer
BIGINT	8 bytes	Large integer
BIT(n)	n bits	Bit field

Integer types can be **signed (default)** or **unsigned** (no negative values, doubles positive range).

Unsigned INT → All bits represent the numeric value.
Example: 11111111 (8 bits) = 255.

Signed INT → The MSB is reserved as the sign bit. MSB = most significant bit (Leftmost bit)
0 in MSB → Positive number
1 in MSB → Negative number (using two's complement representation)



MySQL data types : Numeric : Floating Point

1.Numeric Types (Floating Point)

Approximate precision types:

Data Type	Storage Size	Description
FLOAT	4 bytes	Approximate numeric, single precision
DOUBLE	8 bytes	Approximate numeric, double precision

FLOAT and **DOUBLE** are approximate types; **DOUBLE** provides higher precision than **FLOAT**.
Typically used in scientific calculations or where some precision loss is acceptable.

Exact Precision Numeric Type

- **DECIMAL(m, n)** – Fixed-point exact precision numeric type.
 - m = precision → total number of digits (before + after decimal).
 - n = scale → digits after the decimal point.
 - Example :- DECIMAL(5, 2) so Number will be 123.45
- Used for **storing exact values** such as:
 - Quantities
 - Percentages
 - Monetary values (prices)
- **DECIMAL** is important where **precision matters**, avoiding rounding errors typical with floating-point.



MySQL data types : Datetime types

Data Type	Description	Format	Range
DATE	Stores date only	YYYY-MM-DD	'1000-01-01' to '9999-12-31'
TIME	Stores time only	HH:MM:SS	'-838:59:59' to '838:59:59'
DATETIME	Stores date and time	YYYY-MM-DD HH:MM:SS	'1000-01-01 00:00:00' to '9999-12-31 23:59:59'
TIMESTAMP	Stores timestamp (date + time), time zone aware	YYYY-MM-DD HH:MM:SS	'1970-01-01 00:00:01' UTC to '2038-01-19 03:14:07' UTC
YEAR	Stores year only	2-digit or 4-digit	1901 to 2155 (4-digit year)

Notes:

- **TIMESTAMP** is stored as the number of seconds since the Unix epoch (1970-01-01 00:00:00 UTC).
- **DATETIME** stores values independent of time zone and has a wider range.
- **TIME** can store negative times for durations. Example: A process was supposed to take 2 hours but finished 15 minutes early. You might store -00:15:00 to indicate it was 15 minutes less than planned.
- **YEAR** can be stored as 2-digit or 4-digit format but the actual range is limited.



MySQL data types : String types

- **Storage Size Calculation**
 - "SunBeam" has 7 characters.
In ASCII/Latin1, each character = 1 byte.
Storage = $7 \times 1 = 7$ bytes.
 - The size of string data is calculated as: **Number of characters × size of each character**
- **1. CHAR**
 - **Fixed-length string.** means the database always reserves the same amount of space for every value in that column, no matter how short or long the actual data is.
 - **Length: 1 to 255 characters.** Max = CHAR(255) allowed
 - **Offers very fast access** because the length is constant and predictable. Because the length is constant, the database engine knows exactly where each value starts and ends in memory or on disk. This predictability makes scanning, indexing, and comparing faster.
 - **Pads shorter strings with spaces to fill the fixed length.** CHAR(5) = Insert val 'AB' Stored internally as 'AB ' (3 spaces added).
When retrieved, MySQL may trim trailing spaces automatically depending on context.
- **2. VARCHAR**
 - **Variable-length string.**
 - **Length: 1 to 65,535 characters** (depending on row format and character set). VARCHAR = variable-length string, space-saving, flexible, but needs extra bytes for length info.
 - **Stores actual length + characters** (adds 1 or 2 bytes for length info).
 - **More flexible than CHAR but slightly slower due to variable size management.**
- **3. TEXT Types** (Variable length, generally slower access)

Type	Max Length	Description
TINYTEXT	Up to 255 chars	Small text field
TEXT	Up to 64 KB	Standard text field
MEDIUMTEXT	Up to 16 MB	Larger text field
LONGTEXT	Up to 4 GB	Very large text field



CHAR vs VARCHAR vs TEXT

- **CHAR**

- Fixed inline storage.
- If smaller data is given, rest of space is unused.
- Very fast access.

- **VARCHAR**

- Variable inline storage.
- Stores length and characters.
- Slower access than CHAR.

- **TEXT**

- Variable external storage.
- Very slow access.
- Not ideal for indexing.

- CREATE TABLE temp(c1 CHAR(4), c2 VARCHAR(4), c3 TEXT(4));
- DESC temp;
- INSERT INTO temp VALUES('abcd', 'abcd', 'abcdef');

Feature	CHAR	VARCHAR	TEXT
Type	Fixed-length string	Variable-length string	Variable-length string (large)
Length	1 to 255 characters	1 to 65,535 characters (depends)	Up to GBs (very large)
Storage	Always uses full length (pads with spaces)	Actual characters + 1 or 2 bytes for length info	Stored outside table row, with pointer
Speed	Fast (predictable length)	Slightly slower (must track length)	Slow (external storage access)
Use Case	Fixed codes, flags, IDs	Names, titles, flexible strings	Long text: articles, logs, descriptions



Thank You

